Архитектура высоконагруженных систем: System Design на примере In-Memory кэша

C# Developer. Advanced

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?



Преподаватель



Евгений Волобуев

Full Stack C# .NET Developer

- Ведущий программист компании Simbirsoft
- Более 6 лет в сфере промышленной и коммерческой разработки
- Опыт разработки высоконагруженных и отказоустойчивых систем
- В составе команды создаю мобильный интернет банк для стран латинской америки

@e_volobuev

Правила вебинара

💬 Активно
участвуем

✈️ Off-topic обсуждаем
в учебной группе **OTUS C#-in-de**

✏️ Задаем вопрос
в чат или голосом

⏳ Вопросы вижу в чате,
могу ответить не сразу



Маршрут вебинара

- Введение в System Design
- Что такое кэш
- Проектирование High-Level архитектуры
- Разбор компонент
- Выявление рисков
- Вопросы и ответы

Цели вебинара

К концу занятия вы сможете

1. Декомпозировать сложную задачу на логические компоненты и сервисы

2. Проектировать API и протокол взаимодействия для сетевого сервиса

3. Выбирать подходящие технологии и структуры данных на основе требований

4. Выявлять потенциальные узкие места и точки отказа на этапе проектирования

Смысл

Это нужно уметь чтобы:

1. Осознанно выбирать технологии и структуры данных

2. Проектировать системы, которые не просто работают, а устойчивы к нагрузкам и сбоям

3. Перейти от написания отдельных классов и методов к проектированию полноценных сетевых сервисов

Интерактив

Нарисовать дом за 2 минуты руками другого человека



Анализ результата



Введение в System Design



System Design

— это процесс проектирования сложных программных систем. Это не просто написание кода, а создание "чертежа" всей системы.

- Определение требований
- Разбиение на компоненты
- Выбор технологий
- Проектирование интерфейсов и протоколов
- Анализ производительности и масштабируемости
- Обеспечение надежности и безопасности

Функциональные и Нефункциональные требования

Функциональные

Отвечают на вопрос “**ЧТО** система делает?”

Примеры:

- Пользователь может войти в систему.
- Товар можно добавить в корзину.
- Система отправляет email-подтверждение.

Нефункциональные

Отвечают на вопрос “**КАК** система это делает?”

Примеры:

- Время ответа сервера < 200 мс.
- Система выдерживает 10 000 RPS.
- Доступность сервиса 99.99%.

Шпаргалка по НФТ

Часто используемые группы нефункциональных требований:

- High Availability (Высокая доступность)
- Scalability (Масштабируемость)
- Performance (Производительность)
- Durability (Долговечность)
- Consistency (Консистентность)
- Maintainability (Поддерживаемость)
- Security (Безопасность)
- Cost (Цена)

Сервис кэширования данных

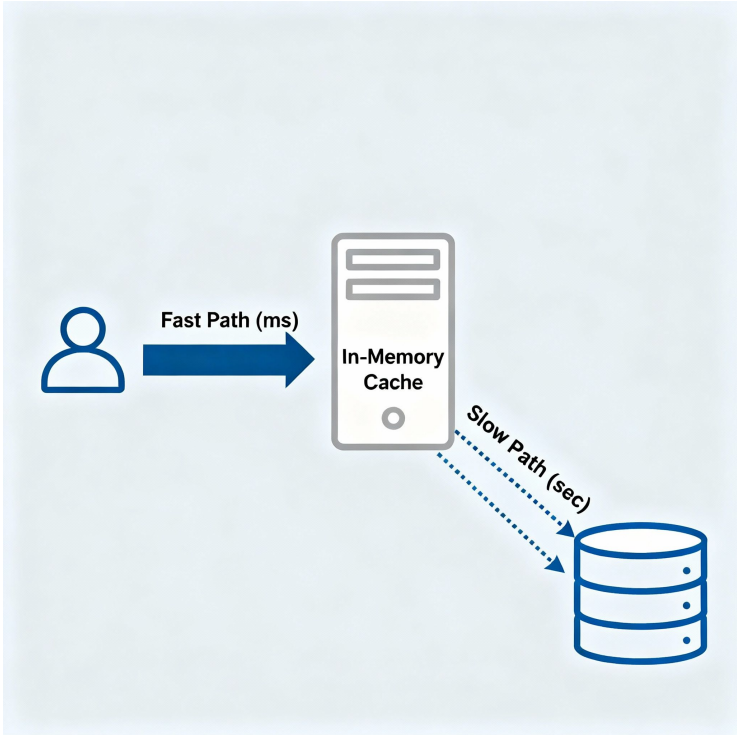


Сервис кэширования

— это сервис, который хранит данные и предоставляет к ним быстрый доступ.

Основная задача:

Ускорить ответ от системы за счет избегания медленных операций (например, чтения с диска или запроса к базе данных).



Интерактив

Напишите в командах ФТ и НФТ к сервису кэширования



Функциональные требования к кэшу

1. Сохранение или обновление значения по строковому ключу.
2. Получение значения по ключу. Если ключа нет, возвращать специальное значение (например, null).
3. Удаление пары "ключ-значение".

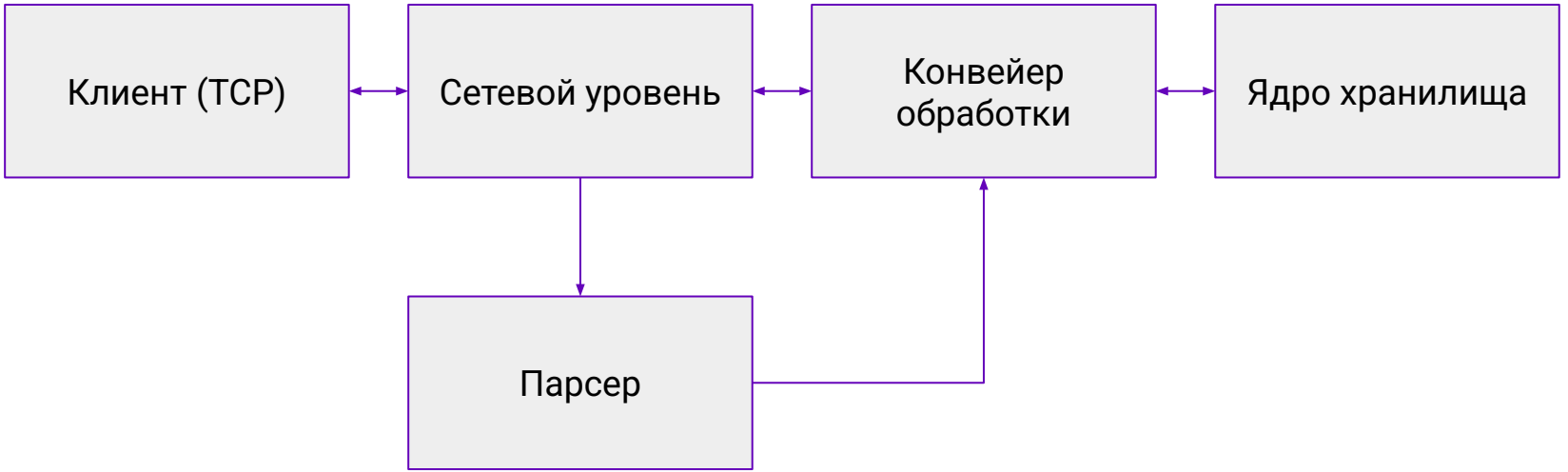
Нефункциональные требования к кэшу

1. Производительность
   - Время ответа на операцию в пределах 100 миллисекунд (p99).
   - Способность обрабатывать не менее 1000 запросов в секунду.
2. Эффективное использование памяти
   - Минимальное количество выделений памяти в управляемой куче на одну операцию (Zero/Low Allocation).
3. Надежность
   - Сервис должен быть потокобезопасным.
   - Сервис не должен падать от некорректных или слишком больших запросов.
4. Наблюдаемость
   - Возможность отслеживать внутреннее состояние: количество сборок мусора (GC), потребление памяти, загрузку CPU, количество операций.

Декомпозиция сервиса



Декомпозиция сервиса кэширования



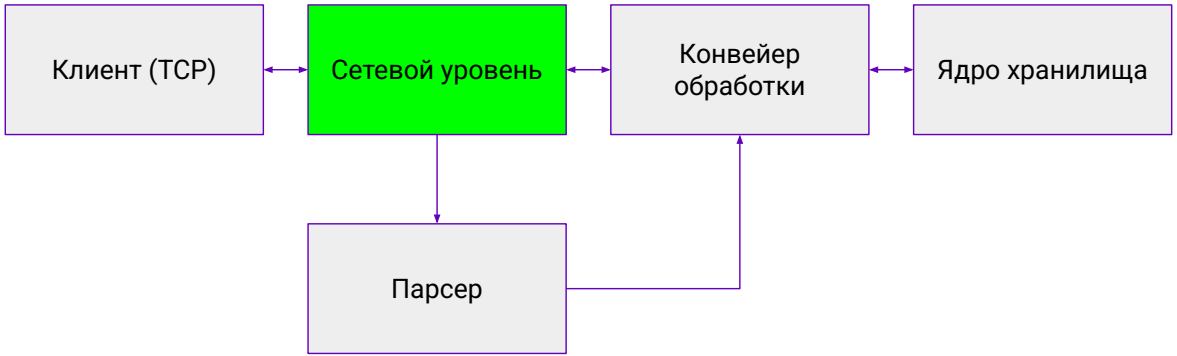
Сетевой уровень



Сетевой уровень

Задача:
Установить "трубу" для передачи байт между клиентом и сервером.

Транспортный протокол:
- **TCP**
- **UDP**
- **HTTP**
- **…**



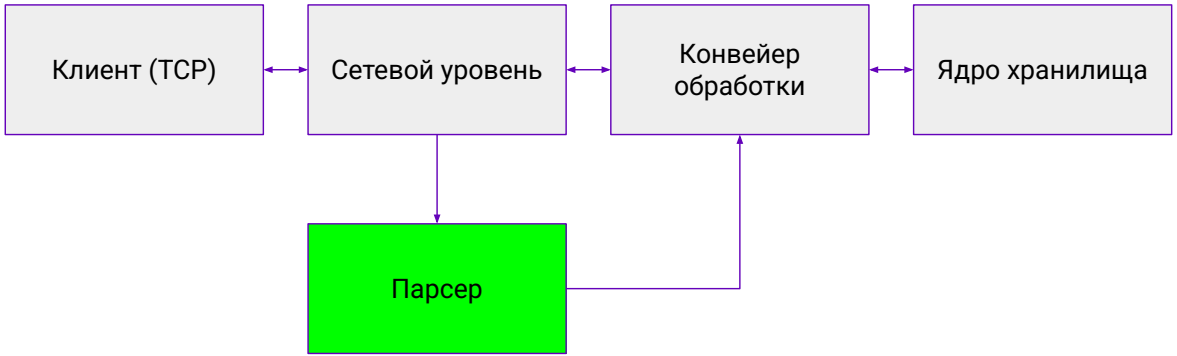
- **TCP:** Гарантирует доставку данных в правильном порядке. Нам не нужно самим реализовывать логику повторных отправок. Идеально для диалога "запрос-ответ". Нет оверхеда на установку соединения для каждой команды.
- **UDP** подходит для потоковой передачи (видео, игры), где потеря пакета не критична. Для кэша это неприемлемо.
- **HTTP** — это протокол более высокого уровня поверх TCP. Он несет много "лишней" информации (заголовки, методы), что для нашего кэша является ненужным оверхедом.

Парсер



Парсер

Задача:
Разобрать что прислал клиент.



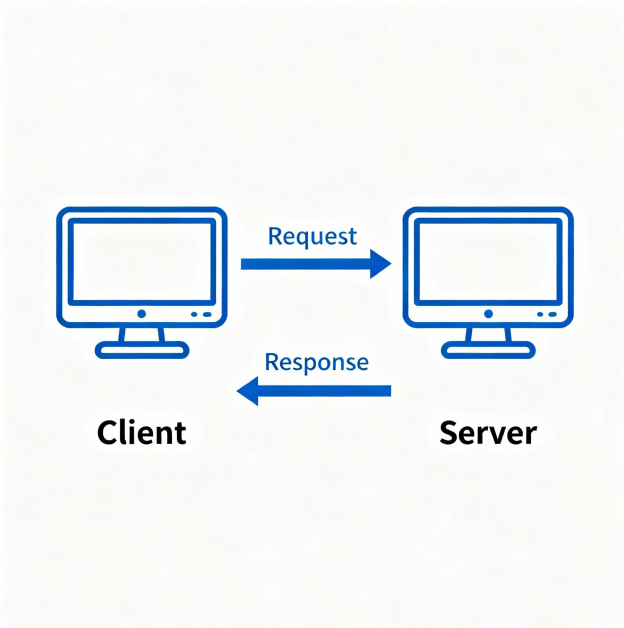
Популярные протоколы:

- **JSON**
- **gRPC**
- **GraphQL**
- **…**

- **Текст:** Простой, быстрый для парсинга (особенно с Span<T>), легко отлаживать. Из минусов: нужно проектировать самим.

Интерактив

Напишите в командах заготовку протокола общения клиента и сервера



Предлагаемый протокол

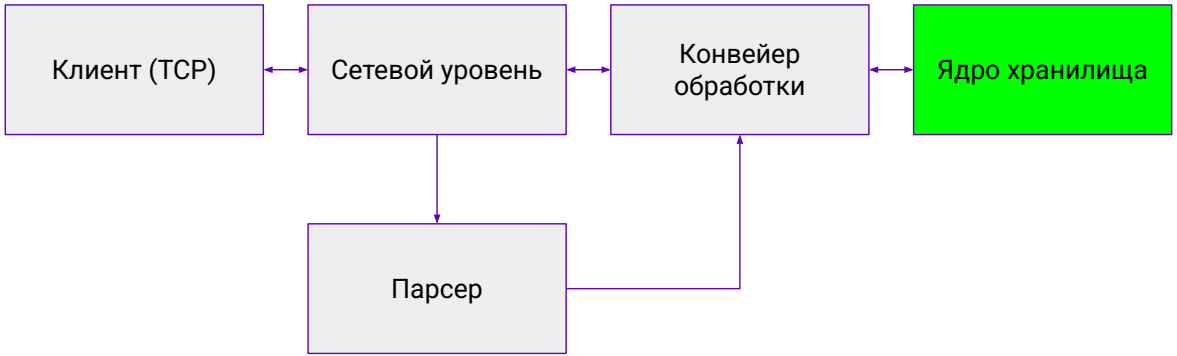
- SET key len value
- GET key
- DEL key
- STA

Ядро хранилища



Ядро хранилища

Задача:
Эффективно хранить пары "ключ-значение" в оперативной памяти и обеспечивать к ним потокобезопасный доступ.



Структуры данных:
- **Dictionary**
- **ConcurrentDictionary**
- **LinkedList**
- **…**

- **Dictionary + ReaderWriterLockSlim:** даёт максимальную производительность для read-heavy нагрузки.

Ядро хранилища

```
private readonly ReaderWriterLockSlim _lock = new();
private readonly Dictionary<string, byte[]> _storage = new();

public byte[] Get(string key)
{
    _lock.EnterReadLock(); // Захват блокировки на чтение
    try
    {
        return _storage.GetValueOrDefault(key);
    }
    finally
    {
        _lock.ExitReadLock(); // Гарантированное освобождение
    }
}
```

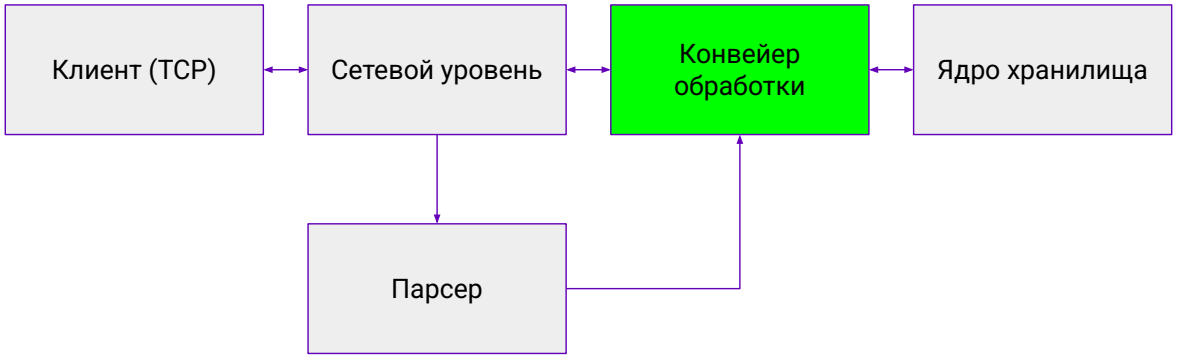
```
public void Set(string key, byte[] value)
{
    _lock.EnterWriteLock(); // Захват блокировки на запись
    try
    {
        _storage[key] = value;
    }
    finally
    {
        _lock.ExitWriteLock(); // Гарантированное освобождение
    }
}
```

Конвейер обработки



Конвейер обработки

Задача:
Связать между собой все компоненты сервиса.



Этот компонент — не отдельный класс, а логика, которая связывает все части вместе.

Конвейер обработки

```
private async Task HandleClientAsync(TcpClient client)
{
    // Бесконечный цикл обработки одного клиента
    while (client.Connected)
    {
        // 1. Сетевой Уровень: прочитать сырые байты
        ReadOnlyMemory<byte> buffer = await ReadDataFromSocket(client);

        // 2. Парсер: превратить байты в команду
        (command, key, value) = CommandParser.Parse(buffer);

        // 3. Конвейер: выбрать действие
        switch (command)
        {
            case "GET":
                // 4. Ядро: выполнить операцию
                var result = _storage.Get(key);
                // 5. Парсер + Сеть: сформировать и отправить ответ
                await SendResponse(client, result);
                break;
            case "SET":
                // ...
                break;
        }
    }
```

Анализ рисков



Анализ рисков

Проектирование — это не только про то, как система работает, но и про то, как она **ломается**.

- **Процессор:**
  - **Сериализация/Десериализация:** Парсинг текста — очень затратная операция.
  - **Конкуренция за блокировки:** Если операции под `WriteLock` будут долгими, вся система встанет.
- **Память:**
  - **Давление на GC:** Бесконтрольное создание буферов и строк приведет к частым и долгим сборкам мусора.
  - **DoS-атака:** Что если клиент пришлет запрос на 10 ГБ?
- **Сеть (Network):**
  - **"Медленный" клиент:** Клиент, который очень медленно читает ответ, может "подвесить" ресурсы сервера.
  - **DoS-атака:** Что если к нам одновременно подключатся 100 000 клиентов?
- **Надежность:**
  - **Единая точка отказа (SPOF):** Наш сервер один. Если он падает — все данные и сервис теряются. (В реальном мире решается репликацией и кластеризацией).

Вопросы

+ если есть вопросы

– если вопросов нет

Подведём итоги занятия

Цели вебинара

К концу занятия вы сможете

1. Декомпозировать сложную задачу на логические компоненты и сервисы

2. Проектировать API и протокол взаимодействия для сетевого сервиса

3. Выбирать подходящие технологии и структуры данных на основе требований

4. Выявлять потенциальные узкие места и точки отказа на этапе проектирования



Отправьте в чат эмодзи, который отражает ваше настроение после занятия



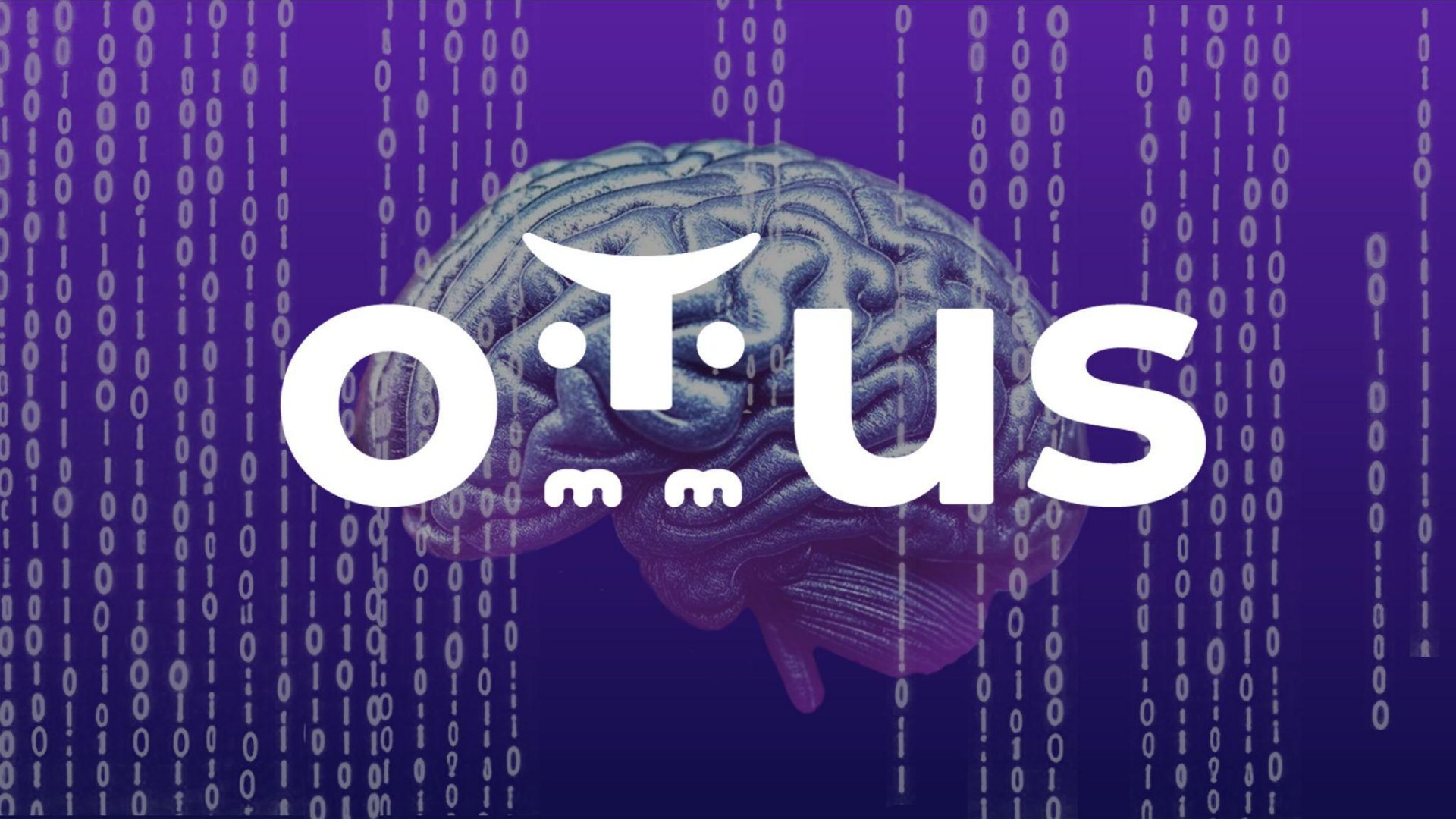
Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️


OTUS